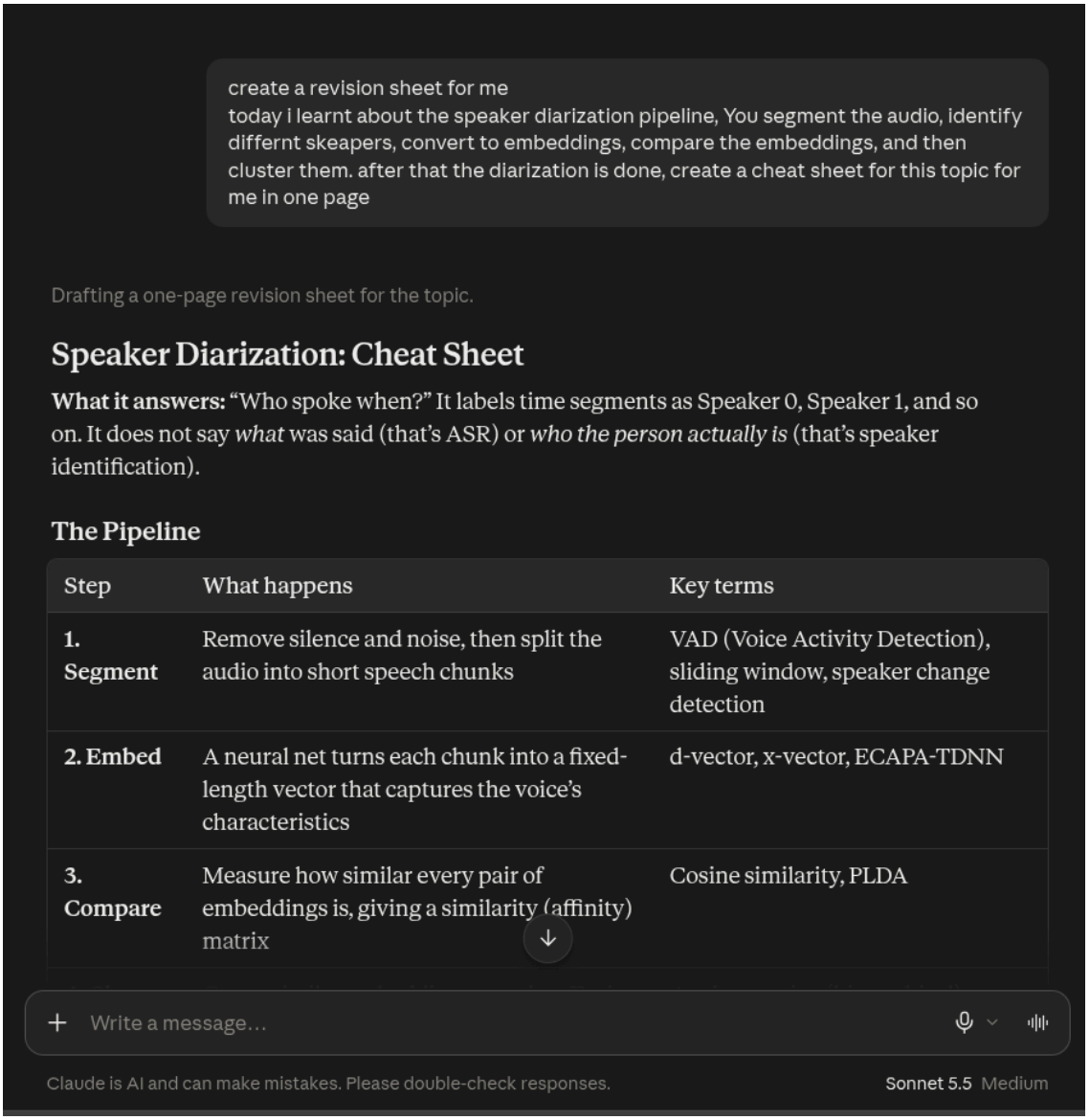
Skill: This skill creates simple one page notes of a given topic. It could be a topic of concept I learnt on a given day. It organizes the information cleanly and concisely on a single page in a story format.

Before:



Core Intuition

- Same voice → embeddings land **close together** in vector space.
- Different voices → embeddings land **far apart**.
- Clustering finds those groups **without knowing speaker identities in advance** (unsupervised).

Key Concepts

- **Embedding:** a compact numerical "voiceprint" of a segment.
- **Cosine similarity:** the angle between two vectors. Closer to 1 means more likely the same speaker.
- **Number of speakers:** often unknown, so clustering uses a threshold or an estimate (e.g. eigengap in spectral clustering).
- **Resegmentation (optional):** refines boundaries after clustering.

Evaluation

DER (Diarization Error Rate) = Missed speech + False alarm + Speaker confusion. Lower is better.

Common Challenges

- Overlapping speech (two people talking at once)
- Very short segments, which give weak embeddings
- Background noise or similar-sounding voices
- Unknown or large number of speakers

Popular Tools

`pyannote.audio`, NVIDIA `NeMo`, `Resemblyzer`, `SpeechBrain`

Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Sonnet 5.5 Medium

After:

Tuesday, 6 October 2026

Who Spoke When: Sorting Voices

A study day spent learning how a machine listens to a messy conversation and works out who said what.

KEYWORDS

Speaker diarization
Answering 'who spoke when' in an audio recording.

Segmentation
Cutting audio into short chunks, each with one speaker.

Speaker embedding
A vector that captures the 'fingerprint' of a voice.

Similarity
How close two embeddings are, e.g. cosine similarity.

Clustering
Grouping similar embeddings; each group is one speaker.

Speaker label
Tag like Speaker 1 or Speaker 2 for each cluster.

TODAY'S STORY

Today I learned how a computer figures out **who spoke when**, which is what **speaker diarization** does. I imagined a noisy group call where everyone talks over each other, and the machine has to untangle it.

First comes **segmentation**: the audio is sliced into small pieces, each ideally holding just one voice. Then every piece is turned into a **speaker embedding**, a list of numbers acting like a voice fingerprint.

Next, the pipeline compares embeddings using a **similarity** score. Pieces that sound alike score high. Finally **clustering** groups them, and each cluster becomes one person with a **speaker label**. Once the clusters are formed, the diarization is done. Neat: nobody had to tell the system how the speakers sound beforehand.

QUICK SELF-TEST

1. **What question does diarization answer?**
   → *Who spoke when*
2. **What are the 4 pipeline steps in order?**
   → *Segment, embed, compare, cluster*
3. **Why convert audio chunks to embeddings?**
   → *To compare voices numerically*
4. **What does each final cluster represent?**
   → *One speaker*

REMEMBER

- Audio → segments → embeddings → similarity → clusters → labels
- Cosine similarity: $\cos(\theta) = (A \cdot B) / (|A| \times |B|)$
- Similar voices → close embeddings → same cluster
- Output = speaker label + start time + end time